

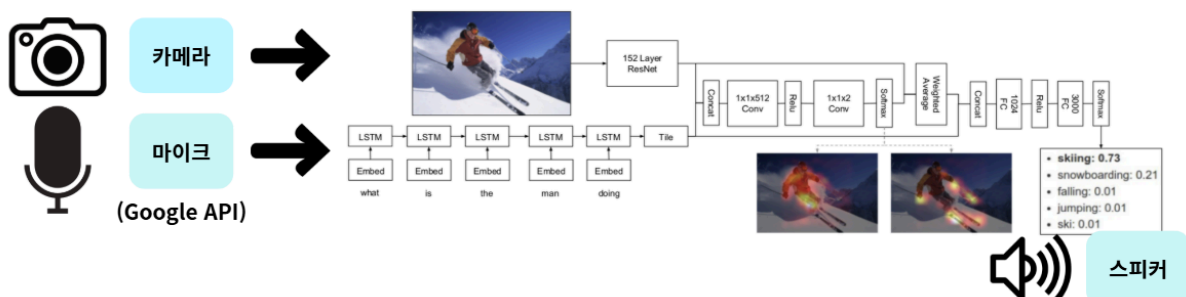
EAI_Final_Team_7

20231171 황민아, 20231377 오규안

1. Task

Visual Question Answering (VQA) 태스크를 리소스가 제한된 Jetson Nano 환경에서 실시간으로 구현하는 것을 목표로 task를 삼아 임베이드 프로젝트를 진행하였습니다. 기존의 단순 이미지 분류와 달리, VQA는 이미지와 자연어 질문을 동시에 처리해야 하므로 모델의 크기가 방대합니다. 이를 구현하기 위해서 ResNet152 기반의 강력한 백본을 유지하면서도, Structural Pruning, Knowledge Distillation, FP16 Quantization의 3단계 경량화 파이프라인을 적용하여, 다양한 질문에 답변 가능한 General VQA 모델을 만들고, 다양한 경량화 기법들을 도입하여 성능과 속도 측면의 비교를 진행해보았습니다.

< 베이스 모델 선정 > : Show, Ask, Attend, and Answer: A Strong Baseline For Visual Question Answering



1) 이미지 인코더: ResNet152를 백본으로 사용하여 이미지가 담고 있는 시각적 정보를 추출합니다. 단순한 분류를 넘어 '무엇이 어디에 있는지' 파악하기 위해 마지막 합성곱 피쳐 맵을 사용하며, 최종적으로 $14 \times 14 \times 2048$ 과 같은 크기의 3차원 텐서를 출력하여 공간적 정보를 유지합니다.

2) 텍스트 인코더: 사용자의 질의 의도를 정확히 이해하기 위해 LSTM 네트워크를 활용합니다. 질문 문장을 임베딩 벡터로 변환한 뒤 Multi-layer LSTM을 통과시키며, 이때 산출되는 마지막 시점의 Hidden State를 해당 질문을 대표하는 핵심 벡터로 사용합니다.

3) 이렇게 ResNet152와 LSTM 을 토대로 얻어진 이미지 특징과 질문 특징을 결합하여 질문에 대한 답을 추론하게 되는데, 먼저 1차원 형태인 질문 정보를 이미지 크기에 맞게 여러 개로 복사하여, 이미지의 모든 위치에서 질문 내용을 참고할 수 있도록 만듭니다. 그 다음, 모델이 질문을 해결하려면 이미지의 어느 부분을 집중해서 봐야 하는지를 스스로 계산하여 Attention Map을 생성합니다. 이 지도를 바탕으로 불필요한 배경 정보는 억제하고

정답의 단서가 되는 핵심 정보만 추출한 뒤, 최종적으로 후보 단어 중 가장 확률이 높은 정답을 예측하도록 추론이 진행이 됩니다.

2. 코드 및 실행 모델

(1) Base Model

(2) Base Model + ResNet Structural Pruning (50%)

(3) Base Model + ResNet Structural Pruning (50%) + Knowledge Distillation

(4) Base Model + ResNet Structural Pruning (50%) + Knowledge Distillation + FP16 Quantization

```
def run_once_bench(  
    model,  
    img,  
    q,  
    q_len,  
    label,  
    override_dtype=None,  
    warmup=5,  
    repeat=20,  
):
```

- 정확한 성능 비교를 위해 단일 추론이 아닌 반복 추론의 평균을 측정하는 벤치마크 함수를 구현했습니다.

`run_once_bench` 함수는 `warmup` 과정을 거친 후, `repeat=20` 설정에 따라 추론을 20회 반복하고 그 평균 시간을 계산합니다.

2 - (1) Base Model

```
# 2) 모델 정의  
class ResNetLayer4(nn.Module):  
    def __init__(self):  
        super().__init__()  
        # GPU 학습을 위해 pretrained 모델 로드  
        self.r_model = resnet152(pretrained=True)  
  
    def forward(self, x):  
        x = self.r_model.conv1(x)  
        x = self.r_model.bn1(x)  
        x = self.r_model.relu(x)  
        x = self.r_model.maxpool(x)  
        x = self.r_model.layer1(x)  
        x = self.r_model.layer2(x)  
        x = self.r_model.layer3(x)  
        x = self.r_model.layer4(x)  
        return x
```

```

class VQAResNetModel(Net):
    def __init__(self, num_tokens):
        super().__init__(num_tokens)
        self.resnet_layer4 = ResNetLayer4()

    def forward(self, v, q, q_len):
        q = self.text(q, list(q_len.data))
        v = self.resnet_layer4(v)
        v = v / (v.norm(p=2, dim=1, keepdim=True).expand_as(v) + 1e-8)
        a = self.attention(v, q)
        v = apply_attention(v, a)
        combined = torch.cat([v, q], dim=1)
        answer = self.classifier(combined)
        return answer

```

(1) ResNetLayer4 : Visual Encoder

- **Pretrained ResNet152** 활용:
`self.r_model = resnet152(pretrained=True)` 코드를 통해, 이미 대규모 데이터셋(ImageNet)으로 학습된 **ResNet152** 모델을 불러옵니다. 이는 이미지 내의 복잡한 객체와 패턴을 인식하는 능력이 매우 뛰어나지만, 연산량이 많고 무거우므로, 경량화가 필요한 대상으로 잡았습니다..

(2) VQAResNetModel :VQA 전체 모델 구조 (Integration & Inerence)

- 멀티모달 입력 처리:
`forward(self, v, q, q_len)` 함수는 이미지(**v**)와 질문(**q**)을 동시에 입력받습니다.
 - 질문 처리: `self.text(q, ...)`를 통해 자연어 질문을 벡터 형태로 변환합니다.
 - 이미지 처리: 앞서 정의한 `self.resnet_layer4(v)`를 호출하여 이미지 특징을 추출합니다.
- **Attention** 메커니즘 적용:
`a = self.attention(v, q)`와 `apply_attention(v, a)`로, 질문(**q**)의 내용에 따라 이미지(**v**)의 어떤 부분을 집중해서 봐야 할지 계산합니다. (예: "색깔이 뭐야?"라고 물으면 사물의 색상 영역에 집중)
- 최종 답변 도출:
`combined = torch.cat([v, q], dim=1)`을 통해 이미지 정보와 질문 정보를 하나로 합치고, `self.classifier`를 거쳐 최종 정답을 출력하도록 설계가 되어있습니다.

2 - (2) Base Model + ResNet Structural Pruning (50%)

: L1 Norm 기반 구조적 프루닝(Structured Pruning)을 통해 모델의 필터를 제거하여 실제 연산량과 메모리를 감소시키는 기법입니다.

Filter Pruning

- L1 Norm을 기준으로 각 Filter의 절댓값 크기를 기반으로 중요도를 평가하여, ranked를 한 뒤에, 하위 50% Filter 제거했습니다.

- Bottleneck 구조는 중간 채널 폭이 크기 때문에, 중요도가 낮은 필터를 제거하면 연산량과 파라미터가 크게 감소합니다.

inference_4models.py

- prune_bottleneck_layer

```
# 2) Pruning 함수 (0.5 Uniform)
def prune_bottleneck_layer(block, prune_ratio=0.5):
    w = block.conv1.weight.data
    importance = torch.sum(torch.abs(w), dim=(1, 2, 3))
    num_total = w.shape[0]
    num_keep = int(num_total * (1 - prune_ratio))
    if num_keep < 1: return

    _, keep_indices = torch.topk(importance, num_keep)
    keep_indices, _ = torch.sort(keep_indices)

    old_conv1 = block.conv1
    new_conv1 = nn.Conv2d(old_conv1.in_channels, num_keep, kernel_size=1, stride=old_conv1.stride, padding=0, bias=(old_conv1.bias is not None))
    new_conv1.weight.data = old_conv1.weight.data[keep_indices]
    if old_conv1.bias is not None: new_conv1.bias.data = old_conv1.bias.data[keep_indices]

    old_bn1 = block.bn1
    new_bn1 = nn.BatchNorm2d(num_keep)
    new_bn1.weight.data = old_bn1.weight.data[keep_indices]
    new_bn1.bias.data = old_bn1.bias.data[keep_indices]
    new_bn1.running_mean = old_bn1.running_mean[keep_indices]
    new_bn1.running_var = old_bn1.running_var[keep_indices]

    old_conv2 = block.conv2
    new_conv2 = nn.Conv2d(num_keep, num_keep, kernel_size=3, stride=old_conv2.stride, padding=1, bias=(old_conv2.bias is not None))
    temp = old_conv2.weight.data[keep_indices]
    new_conv2.weight.data = temp[:, keep_indices]
    if old_conv2.bias is not None: new_conv2.bias.data = old_conv2.bias.data[keep_indices]

    old_bn2 = block.bn2
    new_bn2 = nn.BatchNorm2d(num_keep)
    new_bn2.weight.data = old_bn2.weight.data[keep_indices]
    new_bn2.bias.data = old_bn2.bias.data[keep_indices]
    new_bn2.running_mean = old_bn2.running_mean[keep_indices]
    new_bn2.running_var = old_bn2.running_var[keep_indices]

    old_conv3 = block.conv3
    new_conv3 = nn.Conv2d(num_keep, old_conv3.out_channels, kernel_size=1, bias=(old_conv3.bias is not None))
    new_conv3.weight.data = old_conv3.weight.data[:, keep_indices]
    if old_conv3.bias is not None: new_conv3.bias.data = old_conv3.bias.data

    block.conv1 = new_conv1
    block.bn1 = new_bn1
    block.conv2 = new_conv2
    block.bn2 = new_bn2
    block.conv3 = new_conv3
```

: 각 Convolution 레이어 가중치(**weight.data**)의 L1 Norm(절댓값의 합)을 계산하여 필터의 중요도(**importance**)를 정의합니다. 설정된 **prune_ratio** (50%)에 따라 하위 50%의 필터를 제거하고, 중요도가 높은 상위 필터의 인덱스(**keep_indices**)만을 추출하도록 설계했습니다.

- apply_structured_pruning

```
def apply_structured_pruning(model, ratio=0.5):
    print(f"[*] Applying Structured Pruning (Uniform Ratio: {ratio})...")
    count = 0
    for m in model.modules():
        if isinstance(m, Bottleneck):
            prune_bottleneck_layer(m, ratio)
            count += 1
    print(f"[*] Pruned {count} blocks.")
```

- ResNet152의 모든 Bottleneck 블록에 적용하였으며 (총 50개 블록), Conv1, BN1, Conv2, BN2, Conv3 레이어를 모두 재구성이 되도록 **pruning** 코드를 작성하였습니다. 따라서, 파라미터량 메모리 사이즈, 그리고 속도에 큰 변화를 볼 수 있었습니다.

경량화 효과

Model Name	Inference Time	Memory Size	Parameters #	Answer
Original FP32 Model	8.5s	306.04 MB	80,075,210	Phone
Only Pruned Model	4.6s	172.32 MB	45,043,888	Shadow

: 기본 모델과 비교하여,

- 파라미터 수: 80,075,210 → 45,043,888 (43.7% 감소)
- 메모리 사용량: 306.04 MB → 172.32 MB (43.7% 감소)
- 추론 속도: 851.45 ms → 460.98 ms (1.85배 향상)

이 세가지의 평가 요소에서 향상이 되었지만, **pruning**의 결과 원본과 같이 최적화가 되어있지 않아서, 정확도가 낮은 답변을 하는 경우가 다수 발생 하여 **knowledge distillation**을 통해서 이의 문제를 해결하고자 하였습니다.

2- (3) Base Model + ResNet Structural Pruning (50%) + Knowledge Distillation

Teacher Model : Pruning을 하지 않은 원본 **resnet152** 기반의 VQA 모델

Student Model : 50% Pruning이 된 모델

- VQA v2 Dataset을 기반으로, MS COCO 2014 Validation Set을 활용하여 Distillation을 수행.

distillation.py

- distillation_loss

```
def distillation_loss(student_logits, teacher_logits, labels, T=4.0, alpha=0.5):  
    soft_loss = nn.KLDivLoss(reduction='batchmean')(  
        F.log_softmax(student_logits / T, dim=1),  
        F.softmax(teacher_logits / T, dim=1)  
    )  
    hard_loss = F.cross_entropy(student_logits, labels)  
    return alpha * hard_loss + (1. - alpha) * (T * T) * soft_loss
```

distillation_loss 함수는 두 가지 손실 함수의 가중 합으로 구성됩니다.

1. **Soft Loss:** Teacher 모델의 출력(Softmax 확률 분포)과 Student 모델의 출력 간의 **KLDivLoss**를 계산하여, 정답 외의 클래스 간 관계 정보까지 학습합니다.
Temperature $T=4.0$ 으로 확률 분포를 부드럽게 만들어 클래스 간 관계 정보 학습이 진행이 되었으며, 정답 클래스 뿐만이 아니라 오답 클래스의 상대적인 확률도 학습을 하는 **loss**입니다.
2. **Hard Loss:** 실제 정답 레이블(**labels**)과의 **CrossEntropy** 오차를 계산하여 모델이 정확한 예측을 할 수 있도록 유도를 하는 **loss**입니다.

=> 이 두 손실을 **alpha** 비율로 가중 결합하여($\alpha * \text{hard_loss} + (1-\alpha) * \text{soft_loss}$), 모델 크기는 줄이면서도 원본의 성능을 최대한 유지하도록 재학습했습니다.

- main : distilled 된 파일을 **vqa_pruned_50_distilled_epoch5.pth** 로 저장

```
# 2. Teacher 준비
print("[*] Setting up Teacher...")
teacher = VQAResNetModel(num_tokens)

base_net = Net(num_tokens)
clean_state = {k.replace("module.", ""): v for k, v in saved_state["weights"].items()}
base_net.load_state_dict(clean_state)

t_state = teacher.state_dict()
t_state.update(base_net.state_dict())
teacher.load_state_dict(t_state)

# Multi-GPU 설정
if torch.cuda.device_count() > 1:
    print("[*] Activate DataParallel on (torch.cuda.device_count()) GPUs!")
    teacher = nn.DataParallel(teacher)

teacher.to(device)
teacher.eval()
for p in teacher.parameters():
    p.requires_grad = False

# 3. Student 준비 (Pruning 및 학습 DataParallel)
print("[*] Setting up Student & Pruning 50%...")
student = VQAResNetModel(num_tokens)
student.load_state_dict(t_state)

apply_structured_pruning(student.resnet_layer4.r_model, ratio=0.5)

# Multi-GPU 설정 (Student)
if torch.cuda.device_count() > 1:
    student = nn.DataParallel(student)

student.to(device)
student.train()

# 4. Dataset
transform = transforms.Compose([
    transforms.Resize((448, 448)),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

print("[*] Loading Validation Dataset...")
train_dataset = VQAWallImageDataset[IMG_DIR, Q_PATH, A_PATH, vocab.from_chyt, transform]
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=8, pin_memory=True)

# 5. Training
optimizer = optim.Adam(student.parameters(), lr=LR)
```

: Teacher은 frozen 된 상태로, Student 모델을 학습 시켰습니다.

Knowledge Distillation의 효과

Model Name	Inference Time	Memory Size	Parameters #	Answer
Original FP32 Model	8.5s	306.04 MB	80,075,210	Phone
Only Pruned Model	4.6s	172.32 MB	45,043,888	Shadow
Pruned Model with KD	4.6s	172.32 MB	45,043,888	Phone

: 기본 모델, 기본 + Pruning 모델과 비교하여,

- Pruning된 상태의 모델과 추론 속도/메모리 변화가 없음 (구조 동일)
- 학습을 통한 정확도 회복
- Teacher의 지식 전이로 성능 저하 최소화

따라서, Knowledge Distillation의 결과, 성능이 다시 원본처럼 좋아졌고, 적은 메모리와 파라미터로 더 효율적이게 되었음을 확인 가능했습니다.

2 - (4) Base Model + ResNet Structural Pruning (50%) + Knowledge Distillation

FP16 Quantization

: 데이터의 표현 정밀도를 32-bit Float에서 16-bit Float로 낮추어 메모리 대역폭을 절약하고 연산 속도를 높이는 기법을 적용하여 quantization을 진행했습니다.

```
print("[*] Building pruned+distilled model...")
model = VQAResNetModel(num_tokens)
safe_load_state_dict(model, clean_original)
apply_structured_pruning(model.resnet_layer4.r_model, ratio=PRUNE_RATIO)
safe_load_state_dict(model, clean_distilled)

print("[*] Casting to FP16 and exporting...")
model = model.half()
fp16_state = {k: v.cpu() for k, v in model.state_dict().items()}
torch.save(fp16_state, FP16_OUTPUT)
print(f"[✓] Saved FP16 checkpoint to: {FP16_OUTPUT}")
```

`model.half()` 명령어를 통해 PyTorch의 모든 파라미터를 `torch.float32` → `torch.float16`으로 변환했습니다. 이후 `fp16_state` 사전에 변환된 가중치를 저장하여 추론 시 메모리 사용량을 절반 수준으로 감소시켰습니다.

경량화 효과

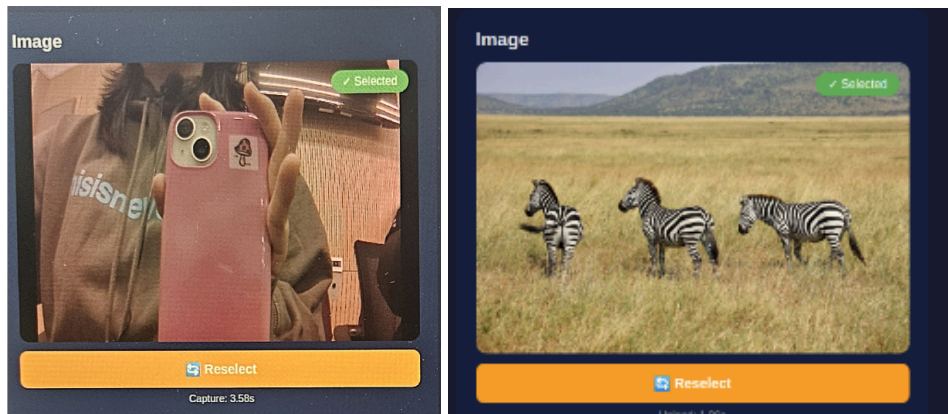
Model Name	Inference Time	Memory Size	Parameters #	Answer
Original FP32 Model	8.5s	306.04 MB	80,075,210	Phone
Only Pruned Model	4.6s	172.32 MB	45,043,888	Shadow
Pruned Model with KD	4.6s	172.32 MB	45,043,888	Phone
Pruned Model with KD and Quantization	4.8s	86.10 MB	45,043,888	Phone

: 최종적으로, final model 에 대해서는 추가적인 memory size에 대한 큰 축소를 할 수 있었으며,

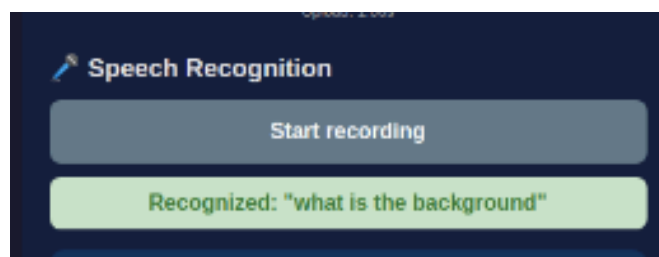
inference time, parameter 수, 그리고 정답까지 맞추며 전반적인 성능을 크게 향상시켰습니다.

3. 모델 비교 및 실제 demo 결과

< Demo details > : Flask 서버 실행 시 자동으로 HTML 템플릿을 생성하도록 UI 설계



- (1) **Camera** : Jetson Nano의 하드웨어 가속 기능을 활용하기 위해 GStreamer와 OpenCV를 결합한 이중 캡처 방식을 적용하였으며, 메모리 누수와 충돌을 방지하기 위해, '**Open** → **Capture** → **Release**' 과정을 단일 요청 단위로 수행하는 **Stateless** 방식으로 카메라를 구현하였습니다. 카메라로 캡처한 이미지는 VQA의 input image로 들어가게 됩니다.
카메라 외에도, 이미지를 업로드 할 수 있도록 설계하였습니다.



- (2) **Microphone** : gTTS (Google Text-to-Speech)를 사용하여 답변 텍스트를 MP3로 변환하고, pydub을 통해 이를 재생 가능한 WAV 포맷으로 변환하도록 설계를 하였습니다. 인식이 된 **speech**는 VQA의 **Question**으로 바로 넣어지게 됩니다.

(3) **Speaker** : VQA 모델이 inference 로 도출한 Answer를 사용자가 청각적으로 즉시 인지할 수 있도록, TTS 파이프라인을 구축하였습니다. (play_audio)

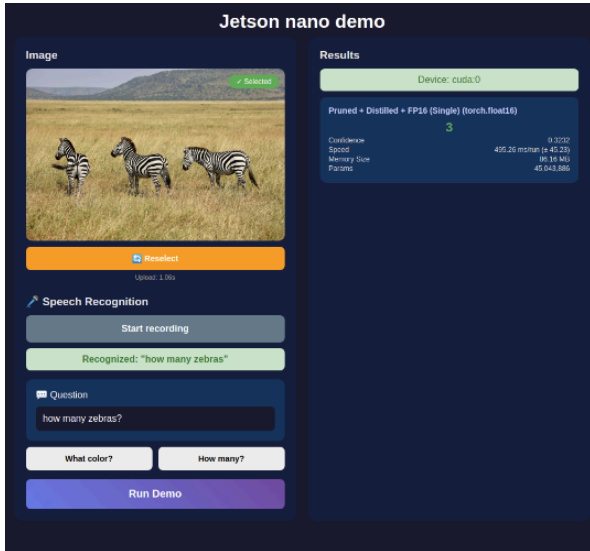
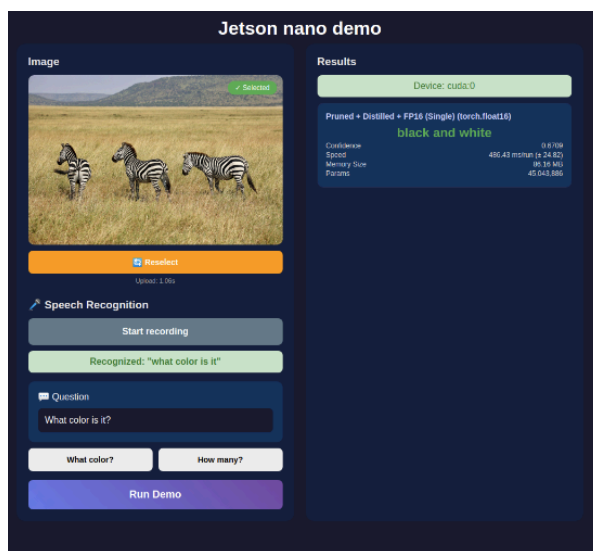
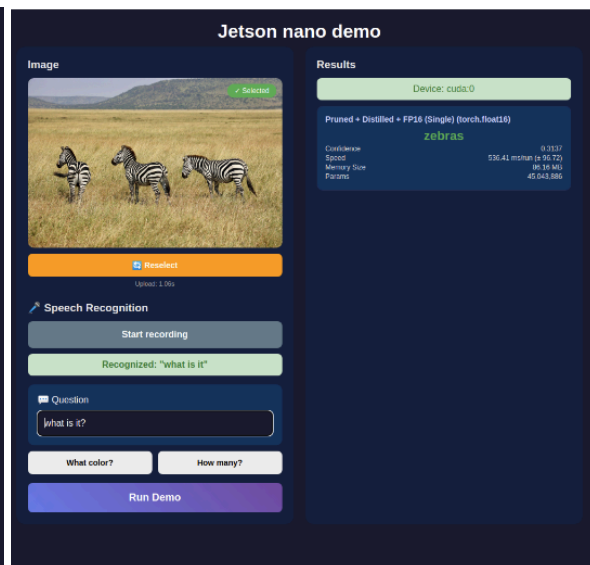
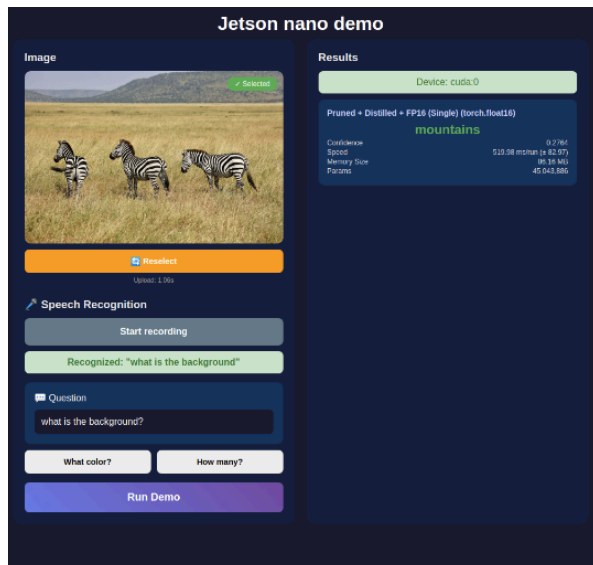
1. 4가지 모델의 성능 비교용의 demo (inference_final_single.py)

The screenshot displays the 'Jetson nano demo' interface. On the left, there's an 'Image' section with a photo of a person holding a pink phone, labeled 'Selected'. Below it is a 'Speech Recognition' section with a 'Start recording' button and a recognized text 'what is this'. At the bottom, there's a 'Question' input field with 'what is this' and buttons for 'What color?' and 'How many?'. A 'Run Demo' button is at the bottom. On the right, the 'Results' section shows performance metrics for four models on a 'Device: cuda:0'.

Model	Confidence	Speed	Memory Size	Params
1. Original FP32 (torch.float32)	0.3899	851.45 ms/run (± 2.39)	306.04 MB	80,075,310
2. Pruned (No Retraining) (torch.float32)	0.1583	460.98 ms/run (± 1.87)	172.32 MB	45,043,886
3. Pruned + Distilled (torch.float32)	0.1480	460.50 ms/run (± 1.23)	172.32 MB	45,043,886
4. Pruned + Distilled + FP16 (torch.float16)	0.1136	483.24 ms/run (± 1.49)	86.16 MB	45,043,886

2. 최종 모델 (inference_final_single.py)

: Original -> Pruned -> Distilled -> Quantized 모델



5. 최종 분석

최종적으로, 정확도와 효율성 자체를 Structural Pruning, Knowledge Distillation, FP16 Quantization의 3단계 파이프라인을 통해 성공적으로 극복하였습니다.

모델 단계	추론 속도	메모리	파라미터 수	비고
1. Original (FP32)	851.45 ms	306.04 MB	80,075,210	기준 모델 (느림)
2. Pruned (50%)	460.98 ms	172.32 MB	45,043,888	속도 약 1.85배 향상, 정확도 하락
3. Pruned + Distilled	460.50 ms	172.32 MB	45,043,888	속도 유지, 정확도 회복
4. Final (FP16)	483.24 ms	86.10 MB	45,043,888	원본 대비 메모리 약 72% 절감, 정확도 유지

Structural Pruning으로 전체 파라미터 수와 추론 지연 시간을 단축시키는 데 성공했지만, 물리적인 필터 제거로 인해 일시적인 정확도 하락 현상이 발생하여 Pruning으로 인한 성능 저하를 보완하기 위해 Knowledge Distillation 기법을 도입했습니다. 이 과정에서 모델의 크기나 추론 속도는 그대로 유지하면서, Teacher 모델의 정보를 통해 정확도를 원본 수준으로 회복시키는 성과를 거두었습니다. 마지막으로, FP16 Quantization까지 적용하여 데이터 표현 대역폭을 축소까지 진행해보았습니다.. 그 결과, 원본(FP32) 대비 메모리 사용량을 306.04MB에서 86.10MB로 약 72% 절감하였습니다. 최종 모델은 초기 모델 대비 절반 수준의 속도와 1/4 수준의 가벼운 메모리로 구동되어, 제한된 리소스

환경에서도 안정적인 **General VQA** 태스크 수행이 가능함을 **demo**를 통해서 확인을 할 수 있었습니다.

단순한 객체 인식을 넘어선 **general** 한 **task**에 대한 **VQA** 추론이 가능한 모델을 완성하였으며, **multi-modal input**에 대한 복합적 추론을 요구하는 "**What is on the background?**"와 같이 객체와 배경 간의 공간적 관계를 이해해야 하거나, "**How many?**"과 같이 객체의 수를 세는 수리적 능력이 요구되는 질문에도 정확한 답변을 생성할 수 있었습니다.